

M.S. Dissertation in Mechanical Engineering

GaussianFeels: Real-Time Multimodal 3D Reconstruction with Tactile-Enhanced Gaussian Splatting

Krishi Attri

Student ID: 2024-24243

Department of Mechanical Engineering

Graduate School

Seoul National University

Spring 2026

Submitted in fulfillment of the requirements
for the degree of Master of Science in Mechanical Engineering

GaussianFeels: Real-Time Multimodal 3D Reconstruction with Tactile-Enhanced Gaussian Splatting

by

Krishi Attri

A Thesis Submitted in Fulfillment of
the Requirements for the Degree of

Master of Science

in Mechanical Engineering

at

Seoul National University

Advisor: Professor Yong-Lae Park

Approved by the Examining Committee:

Chairperson

[TODO: Committee Chair Name, Affiliation]

Member

Professor Yong-Lae Park, Seoul National University

Member

[TODO: Committee Member Name, Affiliation]

Date of Examination: [TODO: Month Day, 2026]

Date of Approval: [TODO: Month Day, 2026]

Abstract

Title: GaussianFeels: Real-Time Multimodal 3D Reconstruction
with Tactile-Enhanced Gaussian Splatting

Author: Krishi Attri (2024-24243)

Department: Department of Mechanical Engineering

Advisor: Professor Yong-Lae Park

Degree: Master of Science

Accurate object reconstruction for robotic manipulation requires more than camera-visible geometry. RGB-D sensing provides dense observations of exposed surfaces, but it cannot recover surfaces hidden by the hand or by fingertip contacts during in-hand manipulation. Tactile sensing measures local contact geometry at those difficult regions, but only over a small contact footprint. This thesis presents GaussianFeels, an online visuo-tactile reconstruction and tracking system built around an object-centric 3D Gaussian representation.

The central claim of the thesis is that an explicit object-centric Gaussian map can serve as a practical shared state for contact-rich manipulation: it can be updated online from synchronized RGB-D, tactile, and proprioceptive observations, tracked directly when object-pose supervision is removed, and exported immediately to downstream manipulation modules. The data considered in this thesis are synchronized manipulation sequences, instantiated here by the FeelSight family of simulation, real-world, and occlusion-focused episodes.

GaussianFeels maintains the object map online with sparse Gaussian updates, contact-aware population management, and occlusion-aware multimodal supervision. Object motion is handled through ground-truth, GT-init, and full SLAM modes. In tracked

modes, the system uses tactile-aware frame-0 seeding, a Theseus-based pose optimizer, a frozen object map, and a consecutive-depth ICP prior to stabilize pose estimation under partial visibility. A parallel manipulation branch supplies a frame-0 image-to-3D prior with Hunyuan3D-2-mini and then progressively replaces generated geometry with real camera and tactile measurements as the episode unfolds.

The experimental program evaluates reconstruction quality, pose-tracking stability, robustness under severe occlusion, and the effect of the frame-0 shape prior on downstream manipulation. Together, the method and evaluation argue that explicit Gaussian representations are not only suitable for real-time visuo-tactile reconstruction, but also for the tracking and manipulation interfaces that contact-rich robotic systems require.

Keywords: 3D Gaussian Splatting, tactile sensing, visuo-tactile fusion, object pose tracking, real-time 3D reconstruction, robot manipulation, SLAM

Student Number: 2024-24243

국문 초록

제목: 촉각 강화 가우시안 스플래팅을 이용한 실시간 다중모달 3차원 재구성

저자: Krishi Attri (2024-24243)

학과: Department of Mechanical Engineering

지도교수: Yong-Lae Park 교수

학위: 공학석사

로봇 조작을 위한 물체 재구성은 카메라가 볼 수 있는 표면만으로는 충분하지 않다. RGB-D 카메라는 노출된 표면을 조밀하게 관측할 수 있지만, 손에 가려진 영역이나 손가락 접촉 부근의 형상은 충분히 복원하지 못한다. 반면 촉각 센서는 그러한 접촉 영역의 국소 형상을 직접 측정할 수 있으나 관측 범위가 좁다. 본 논문은 이러한 한계를 보완하기 위해 물체 중심 3차원 가우시안 표현을 사용하는 온라인 시각-촉각 재구성 및 추적 시스템 GaussianFeels를 제안한다.

본 논문의 핵심 주장은 명시적 물체 중심 가우시안 맵이 접촉이 포함된 조작 상황에서 공통 상태 표현으로 기능할 수 있다는 점이다. 즉, 동기화된 RGB-D, 촉각, proprioception 관측으로부터 온라인 갱신이 가능하고, 물체 자세 supervision이 제거된 경우에도 직접 추적할 수 있으며, 조작 모듈에 즉시 전달할 수 있는 물체 형상 표현으로 사용할 수 있다. 본 논문은 이러한 문제를 FeelSight 계열의 시뮬레이션, 실제 환경, 가림 상황 시퀀스를 통해 구체화한다.

GaussianFeels는 희소 가우시안 업데이트, 접촉 인근 밀도 강화, 가림 인식 가중치를 통해 물체 맵을 온라인으로 유지한다. 물체 운동은 GT, GT-init, SLAM의 세 가지 모드로 처리된다. 추적 모드에서는 Theseus 기반 포즈 최적화, 촉각 정보를 반영한 프레임 0 초기화, 연속 깊이 관측 사이의 ICP 초기화, 고정 물체 맵을 이용해 안정적인 물체 자세 추정을 수행한다. 조작 모듈에서는 별도의 프로세스가 Hunyuan3D-2-mini를 사용하여 프레임 0에서 초기 형상 completion prior를 생성하고, 이후 실제 카메라

및 촉각 관측으로 생성 형상을 점진적으로 대체한다.

실험은 재구성 품질, 물체 자세 추적 안정성, 강한 가림 상황에서의 강건성, 그리고 프레임 0 shape-completion prior가 조작에 미치는 영향을 중심으로 구성된다. 이를 통해 본 논문은 명시적 가우시안 표현이 실시간 시각-촉각 재구성뿐 아니라 추적과 조작 인터페이스에도 적합하다는 점을 보이고자 한다.

주요어: 3차원 가우시안 스플래팅, 촉각 감지, 시각촉각 융합, 물체 자세 추적, 실시간 3차원 재구성, 로봇 조작, SLAM

학번: 2024-24243

Contents

Abstract	iii
국문 초록	v
1 Introduction	1
1.1 Motivation	1
1.2 Problem Formulation and Research Claim	1
1.3 Approach	2
1.4 Contributions	3
1.5 Thesis Organization	4
2 Related Work	6
2.1 Neural Implicit Reconstruction for Manipulation	6
2.2 3D Gaussian Splatting for Robotic Mapping	7
2.3 Vision-Based Tactile Sensing	9
2.4 Visuo-Tactile Gaussian Reconstruction	9
2.5 Pose Tracking for Manipulated Objects	9
2.6 Image-to-3D Priors for Manipulation	10
3 Manipulation Data Setup	11
3.1 Sensor Modalities	12
3.2 Dataset Variants	12
3.3 Problem Definition in the FeelSight Setting	13
3.4 Execution Modes	13
3.5 Summary	14
4 GaussianFeels SLAM	15
4.1 System Overview	15

4.2	Sensor Frame Construction	17
4.3	Pose Tracking Inside the SLAM Loop	18
4.4	Object-Centric Gaussian Map	22
4.5	Online Training Objective	24
4.6	Runtime Coordination and Outputs	25
4.7	Summary	25
5	Manipulation-Side Completion	27
5.1	Motivation	27
5.2	Parallel Runtime Structure	27
5.3	Frame-0 Prior with Hunyuan3D-2-mini	28
5.4	Progressive Replacement by Real Geometry	31
5.5	Policy-Facing Output	33
5.6	Summary	34
6	Experiments	35
6.1	Evaluation Questions	35
6.2	Evaluation Metrics	35
6.3	Reserved Result Figures	36
6.4	Reserved Result Tables	40
6.5	Frame-0 Prior Selection Results	40
6.6	Reserved Notes	44
7	Conclusion	45
7.1	What This Thesis Establishes	45
7.2	Limitations	45
7.3	Future Work	47
	References	49
A	Default Runtime Configuration	51
A.1	Baseline Optuna Configuration	51
B	Multi-Object Optimization Report	52

C Manipulation Benchmark Summary Placeholder	53
D Selected Implementation Excerpts	54
D.1 Tactile-Aware Frame-0 Translation Seed	54
D.2 Occlusion-Aware Modality Reweighting	55
D.3 Frame-0 Hunyuan Variant Search and Alignment	55
E Policy Output Contract	57

List of Figures

3.1	Synchronized sensing structure used by GaussianFeels. The thesis evaluates this setup mainly through the FeelSight family, but the same data organization also describes matching real robot manipulation rollouts. .	11
4.1	Per-frame data flow of GaussianFeels SLAM. Pose tracking sits in the middle of the loop because it is the transform that links the incoming observations to the object-centric Gaussian state.	16
4.2	The 24-candidate PCA permutation search used for frame-0 rotation seeding. Each cell represents one axis-aligned $\text{SO}(3)$ candidate R_k , formed by choosing one of the six permutations of the three PCA eigenvectors (v_1, v_2, v_3) (rows) and one of the four consistent sign combinations (columns; the third axis sign is fixed by the right-hand rule). The highlighted cell is the selected candidate R_0 , which minimizes the five-term cost $\phi(R_k)$	20
4.3	Illustration of the Gaussian-density signed-distance field used by the frozen object map. Blue dots are anchor points q_i sampled on the object surface with associated outward normals. For a query point p (red), Gaussian weights w_i fall off with distance (dashed circles). The weighted centroid $\tilde{q}(p)$ and blended normal $\tilde{n}(p)$ define a local tangent plane, and the signed distance $d_{\mathcal{M}}(p)$ is the projection of $p - \tilde{q}$ onto $\tilde{n}$	21
4.4	Occlusion-aware weight schedule. As the occlusion ratio ρ_{occ} increases, the visual weight multiplier s_{vis} decreases linearly from 1 to w_{min} , while the tactile boost factor $(1 + \beta\rho_{\text{occ}})$ increases. The two curves cross at $\rho_{\text{occ}} = (1 - w_{\text{min}})/\beta$, marking the transition where tactile supervision becomes stronger than the unoccluded visual baseline.	25

5.1	Manipulation-side pipeline. The frame-0 bootstrap produces an initial object estimate, and later updates progressively replace generated geometry with measured geometry.	28
5.2	Schematic of progressive replacement. Orange points are generated prior geometry and blue points are measured geometry. As the episode progresses, measured observations cover and remove generated regions, so the object model becomes increasingly measurement-driven.	32
5.3	Real manipulation-side progression from the saved five-frame sweep and the gap-filler preview. In the first three panels, orange points come from the frame-0 generative prior and green points come from measured geometry accumulated during manipulation. By frame 4, more of the object surface has been replaced by measurements. The final panel shows the gap-filler debug cloud, where blue points are temporary bridge points inserted only as an internal seam-stabilization aid; they are not part of the policy-facing export.	33
6.1	Reserved qualitative reconstruction comparison across the three evaluation settings.	36
6.2	Reserved pose-tracking plots.	36
6.3	Reserved figure for hard-visual-condition experiments.	37
6.4	High-resolution qualitative comparison for a near-convex object trial. Each sheet shows the ground-truth point cloud and the saved model outputs from four fixed viewpoints after normalization for shape-only visual comparison. The row labels include the aligned F-score at 5 mm for that trial. On this peach example, Hunyuan3D-2-mini and FastSAM3D recover the object category most faithfully, while Geco and RGB2Point visibly collapse.	37
6.5	Qualitative comparison for a box-like food container. On this trial, Hunyuan3D-2-mini preserves the elongated can geometry more reliably than the other methods, while several alternatives either over-round the shape or collapse toward a generic support volume.	38

6.6	Qualitative comparison for a structured cuboid object. This example shows why the benchmark cannot be interpreted from F-score alone: TripoSR reaches a higher score than Hunyuan3D-2-mini on this trial but still rounds the object into a generic blob instead of preserving a cuboid shape.	38
6.7	Qualitative comparison for an articulated animal-shaped object. Several methods achieve moderate scores despite losing the leg and trunk structure that would matter for manipulation-aware reasoning.	39
6.8	Reserved figure for the effect of shape priors on manipulation.	39

List of Tables

2.1	Position of GaussianFeels relative to the closest prior systems.	7
3.1	FeelSight variants used by GaussianFeels.	12
3.2	Pose-handling modes used in GaussianFeels.	13
6.1	Reserved table for GaussianFeels reconstruction quality.	42
6.2	Reserved table for pose-tracking accuracy.	42
6.3	Frame-0 shape-prior comparison on the saved 40-trial benchmark. . . .	43
6.4	Reserved runtime table for the final SLAM system.	43
6.5	Reserved table for the effect of shape priors on downstream manipulation.	43
A.1	Default baseline configuration	51
B.1	Summary of the multi-object optimization report bundled with the repository.	52
C.1	Manipulation-prior benchmark summary	53
E.1	Manipulation export files	57

Chapter 1

Introduction

1.1 Motivation

Robotic in-hand manipulation depends on geometric information that is hardest to observe with a camera alone. During grasping and reorientation, the object regions that matter most for control are often the ones hidden by the hand, the fingertips, or self-occlusion. At the same time, vision-based tactile sensors only observe a small contact patch. A practical manipulation system therefore needs both modalities: RGB-D sensing for global coverage and tactile sensing for local contact geometry.

1.2 Problem Formulation and Research Claim

This thesis studies object reconstruction from synchronized manipulation data: external RGB-D observations, fingertip tactile observations converted to contact geometry, and hand proprioception collected while a robot manipulates a single object. Those data are instantiated here by the FeelSight family introduced by NeuralFeels [1], but the problem formulation is broader than one benchmark name. The same sensing structure also describes real robot rollouts with equivalent hardware.

The research question is whether one explicit object-centric representation can play all of the roles that contact-rich manipulation needs at once: online reconstruction state, pose-tracking reference, and manipulation-facing object estimate.

The argument of the thesis follows a simple progression.

1. Prior visuo-tactile systems established why in-hand manipulation needs both

global vision and local contact geometry.

2. GaussianFeels keeps that sensing problem, but replaces a neural implicit reconstruction stack with an explicit object-centric Gaussian representation.
3. The Gaussian map is maintained online inside a SLAM-style runtime that couples sensing, pose tracking, and multimodal optimization.
4. A parallel manipulation module supplies an early object estimate from frame 0 and then yields to real geometry as measurements accumulate.

The central technical problem addressed here is therefore:

Given synchronized RGB-D, tactile, and proprioceptive observations of a manipulated object, maintain an object-centric shape representation online, estimate object pose when supervision is unavailable, and expose a useful object model to manipulation from the beginning of the episode.

The thesis claim is:

An explicit object-centric Gaussian representation is a suitable shared state for contact-rich manipulation because it can be updated online from synchronized multimodal observations, tracked directly under partial visibility, and exposed immediately to manipulation as a progressively improving object model.

1.3 Approach

GaussianFeels uses 3D Gaussian Splatting [2] as the shared object representation. The runtime is organized around four tightly connected parts that all support the reconstruction goal of the thesis.

1. Sensor ingestion and preprocessing. RGB-D frames, segmentation masks, camera poses, tactile contact geometry, and hand state are brought into a synchronized per-frame representation. This gives every downstream stage the same geometric view of the episode.

2. Object-centric Gaussian mapping. The map is stored in object coordinates, populated from both camera and tactile evidence, pruned or frozen under a training budget, and synchronized to the current world-frame pose only when rendering or supervision requires it.

3. Pose tracking inside the SLAM loop. The tracker estimates object pose in GT, GT-init, and SLAM modes. It uses tactile-aware frame-0 seeding, image-space depth sampling, a Theseus-based optimizer, a frozen object map, and an ICP prior between consecutive depth observations.

4. Manipulation-side shape completion. A parallel process uses Hunyuan3D-2-mini [3] to create a frame-0 shape prior, aligns that prior to the real frame-0 observation, and then progressively replaces generated geometry with measured geometry as the episode continues.

1.4 Contributions

The contribution list below focuses on implemented technical ideas. Empirical findings are reserved for Chapter 6 after the final experiment sweep is complete.

The main contributions presented in this thesis are:

- 1. An object-centric Gaussian reconstruction backbone for contact-rich manipulation.** GaussianFeels keeps the manipulated object as an explicit Gaussian map whose source-of-truth geometry lives in the object frame and is synchronized to the world frame only when rendering, supervision, or export requires it.
- 2. A pose tracker embedded inside the reconstruction loop.** The tracker supports GT, GT-init, and SLAM modes and combines tactile-aware frame-0 seeding, frozen-map signed-distance residuals, a Theseus-based SE(3) optimizer, and a consecutive-depth ICP prior.
- 3. Contact-aware Gaussian population management.** The runtime allocates

its active Gaussian budget through multimodal spawning, direct contact-point insertion, tactile-region density boosts, and selective freezing so that contact-critical regions remain represented under partial visibility.

4. **An occlusion-aware multimodal training objective.** The reconstruction loss reweights RGB, depth, and tactile supervision online when hand-induced occlusion reduces the reliability of visual evidence.
5. **A dual-process reconstruction-to-manipulation interface.** Reconstruction and manipulation run in parallel: the SLAM loop maintains the object map online while a separate branch prepares an early object estimate for downstream action.
6. **A deployed frame-0 image-to-3D bootstrap based on Hunyuan3D-2-mini.** The manipulation branch builds the initial shape prior from an exact RGBA crop, an orientation-variant search, and a registration stage that solves the model-to-object transform.
7. **A provenance-preserving update rule for manipulation-facing geometry.** Generated geometry is progressively overwritten by measured geometry, while the manipulation interface retains explicit provenance labels so downstream policies know which surfaces are measured and which remain generated.

1.5 Thesis Organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews prior neural implicit, Gaussian, tactile, tracking, and image-to-3D approaches, and identifies the gap that GaussianFeels addresses.

Chapter 3 explains the synchronized manipulation data used in this thesis, including the simulation, real-world, and occlusion-focused dataset variants and the sensing structure they share.

Chapter 4 presents GaussianFeels SLAM: the runtime flow, the Gaussian map, the pose tracker at the center of that pipeline, the training objective, and the system

outputs.

Chapter 5 describes the manipulation-side shape-completion module built around Hunyuan3D-2-mini and the real-data replacement process.

Chapter 6 defines the experimental protocol and reserves the figures and tables where the final results will be reported.

Chapter 7 returns to the research claim and summarizes what the thesis establishes about explicit Gaussian representations for visuo-tactile manipulation.

Chapter 2

Related Work

The literature relevant to GaussianFeels falls into six groups: neural implicit visuo-tactile reconstruction, Gaussian splatting for online mapping, vision-based tactile sensing, visuo-tactile Gaussian reconstruction, pose tracking for manipulated objects, and single-image image-to-3D priors. This chapter reviews those groups in that order and uses them to answer two questions: what technical ingredients already exist, and what combination is still missing for contact-rich object reconstruction during manipulation.

The organizing logic is simple. Some prior systems solve in-hand visuo-tactile tracking and reconstruction, some solve online Gaussian mapping, some show that tactile sensing can supervise Gaussian splatting, and some provide single-image shape priors. The gap addressed by GaussianFeels is the combination: one object-centric state that is explicit enough to render and export, online enough to update under contact, trackable under heavy occlusion, and useful to manipulation from the beginning of the episode.

2.1 Neural Implicit Reconstruction for Manipulation

Neural radiance fields (NeRFs) showed that scene geometry and appearance can be represented by a neural implicit function optimized from images alone [4]. NeuralFeels [1] extended that family of ideas to visuo-tactile in-hand manipulation by combining RGB-D and DIGIT sensing in the FeelSight setting. NeuralFeels is the most relevant prior system for this thesis because it established the visuo-tactile reconstruction and tracking problem in the same general setting.

Table 2.1: Position of GaussianFeels relative to the closest prior systems.

Method	T	O	Tr	E	Main gap relative to this thesis
NeuralFeels	✓	✓	✓	–	implicit field rather than an explicit Gaussian state for direct rendering and export
FingerSLAM	✓	✓	✓	–	visuo-tactile in-hand tracking and reconstruction, but no explicit Gaussian state or frame-0 manipulation prior
Touch-GS	✓	–	–	–	tactile-assisted reconstruction, but not in-hand online tracking
Snap-it / Tap-it / Splat-it	✓	–	–	–	tactile-informed reconstruction, but not synchronized manipulation data with online object tracking
Barad et al.	–	✓	✓	–	object-centric tracking without tactile contact geometry or frame-0 completion
Li and Pathak	–	✓	–	–	object-aware manipulation scenes without visuo-tactile in-hand reconstruction
GaussianFeels	✓	✓	✓	✓	unified object-centric reconstruction, tracking, and early-episode shape support

GaussianFeels inherits that sensing problem, but changes the runtime representation. For the goals of this thesis, an explicit representation is better suited than a neural implicit field because the map must be rendered quickly, updated incrementally, exported directly, and handed off to a separate manipulation module without an additional surface-extraction stage. That representation choice is the main architectural break from NeuralFeels.

FingerSLAM [5] is another close prior because it also targets visuo-tactile in-hand object localization and reconstruction. However, its reconstruction state is built by pose estimation and stitching local point cloud observations recovered from tactile sensing, with loop closure in a factor-graph pipeline. GaussianFeels differs at the representation level and at the deployment interface. The central state here is an explicit Gaussian object map rather than a stitched point-cloud history, and that same state is exposed to a parallel manipulation module with a frame-0 prior.

2.2 3D Gaussian Splatting for Robotic Mapping

3D Gaussian Splatting (3DGS) replaced expensive volumetric ray marching with an explicit set of anisotropic Gaussian primitives rendered by differentiable rasteriza-

tion [2]. Compared with NeRF-style rendering, this representation is attractive for online robotics because the primitives are directly editable, visibility can be managed explicitly, and rasterization is fast enough for real-time operation.

Recent work has brought Gaussian representations into SLAM and online mapping. SplatTAM [6], Gaussian Splatting SLAM (“MonoGS”) [7], and Gaussian-SLAM [8] showed that camera-only Gaussian maps can be updated incrementally while preserving high rendering quality. These systems motivate the mapping side of GaussianFeels, but they do not address tactile sensing or manipulation-facing shape estimation.

More closely related to robotics, Barad et al. [9] used an object-centric Gaussian representation for online RGB-D reconstruction and tracking of unknown moving objects, while Li and Pathak [10] used semantic object-aware Gaussians for dynamic robotic manipulation. These works show that Gaussian maps are useful beyond static novel-view synthesis: they can also support online object understanding and action. However, their input assumptions and task structure remain different from GaussianFeels. Neither is built around synchronized manipulation data that combine RGB-D observations, tactile contact geometry, and hand-state information under severe hand-induced occlusion, nor do they target the need for a usable object model from the beginning of the episode.

In representation terms, 3DGS stores geometry and appearance directly in the primitives themselves: each Gaussian carries a mean, scale, orientation, opacity, and color parameters that can be rendered by differentiable rasterization. That explicit parameterization is what makes GaussianFeels well suited to this thesis problem. During in-hand manipulation, the map must be updated quickly, queried under changing object poses, exported directly, and handed to a separate manipulation module without first extracting a surface from an implicit field.

This difference matters because the reconstruction state in GaussianFeels is not only a renderer. It must also be a tracked object model, an optimization target, and a manipulation-facing export. That is the main reason this thesis builds on Gaussian splatting rather than a NeRF-style representation.

2.3 Vision-Based Tactile Sensing

Vision-based tactile sensors such as GelSight and DIGIT measure local contact geometry by imaging gel deformation [11]–[13]. TouchVIT-style tactile depth prediction, trained with TACTO simulation [14], makes those observations usable as dense local surface cues inside a reconstruction pipeline. GaussianFeels follows this line of work by converting tactile images into contact masks, contact points, and contact normals that can directly supervise the Gaussian object map.

2.4 Visuo-Tactile Gaussian Reconstruction

Touch-GS [15] and Snap-it, Tap-it, Splat-it [16] demonstrated that Gaussian splatting can be supervised by tactile cues. These systems move closer to the representation used in this thesis, but the GaussianFeels problem is more specific. Touch-GS uses optical tactile data to improve Gaussian reconstruction quality, including on visually challenging surfaces, while Snap-it, Tap-it, Splat-it studies tactile-informed 3DGS under sparse observations. GaussianFeels instead targets a single manipulated object observed through synchronized RGB-D, tactile, and hand-state streams, with an object-centric online SLAM loop, explicit pose tracking, and a downstream manipulation module that needs an object estimate from frame 0 onward.

The gap is therefore not merely “vision plus touch.” The unresolved question is how to organize those modalities into one online object representation that is simultaneously trainable, trackable, exportable, and useful before the full surface has been explored physically.

2.5 Pose Tracking for Manipulated Objects

Object tracking during manipulation depends on combining geometric constraints, temporal priors, and partial observations. Classical ICP remains a strong tool for aligning consecutive depth observations [17], [18]. Sliding-window nonlinear optimization is then used to stabilize pose estimates under partial visibility and contact.

FingerSLAM [5] is an important reference point here as well. It uses a factor-graph estimator with an explicit loop-closure mechanism to reduce accumulated drift in real-world visuo-tactile object tracking. The problem structure in GaussianFeels is different: the episodes are short, single-object manipulation sequences centered on one evolving object model, so the active tracker is designed around per-frame alignment to a frozen object map plus a consecutive-depth ICP prior rather than around place-recognition loop closure.

GaussianFeels adopts this general strategy, but applies it inside an object-centric Gaussian SLAM pipeline. The pose estimate is not an isolated output; it is the transform that keeps the Gaussian map, the frozen map, the rendered supervision, and the manipulation-side shape estimate in a shared object frame. The setting also differs from large-scale SLAM: these are short, single-object manipulation episodes, so the runtime does not depend on place-recognition loop closure. The dominant tracking problem is repeated partial observation of the same object under contact, not long-horizon relocalization in an environment.

2.6 Image-to-3D Priors for Manipulation

Single-image image-to-3D generation is useful when a manipulation policy needs an object estimate before the robot has explored the whole surface. This thesis uses Hunyuan3D-2-mini [3], [19] as the deployed frame-0 prior. Its role is not to replace SLAM, but to provide an initial object estimate before enough real measurements have accumulated.

The manipulation module in GaussianFeels treats the generative prior as provisional geometry. As camera and tactile measurements accumulate, real observations overwrite generated surfaces. The final system therefore combines a real-time explicit SLAM map with a separate manipulation-side prior rather than choosing between them.

Chapter 3

Manipulation Data Setup

GaussianFeels is evaluated primarily on the FeelSight family used by earlier visuo-tactile work [1]. Each sequence centers on one manipulated object observed through an external RGB-D camera, local tactile sensing at the fingertips, and hand proprioception. The same synchronized sensing structure also matches real robot manipulation rollouts with equivalent hardware, so the problem definition in this chapter is about synchronized manipulation data rather than about one dataset name alone.

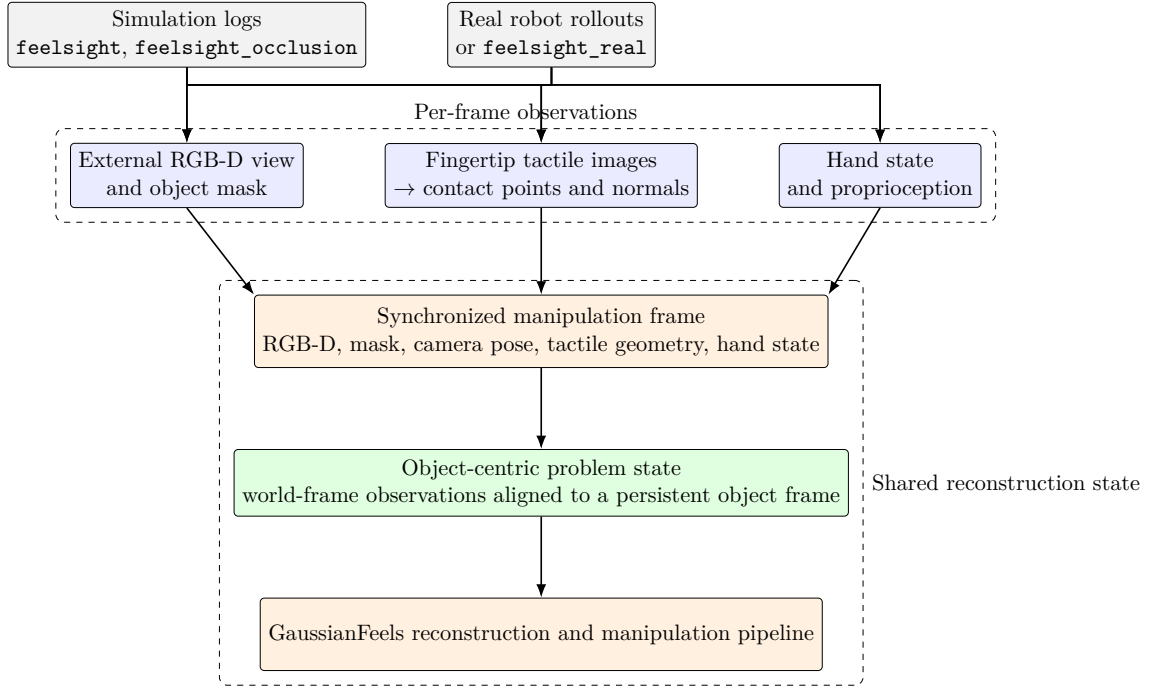


Figure 3.1: Synchronized sensing structure used by GaussianFeels. The thesis evaluates this setup mainly through the FeelSight family, but the same data organization also describes matching real robot manipulation rollouts.

3.1 Sensor Modalities

The implemented system uses the following observations at each frame:

- RGB and depth images from a selected RealSense camera;
- camera intrinsics and per-frame camera pose;
- object pose annotations when the sequence is evaluated in GT mode;
- DIGIT tactile images converted into contact points and contact normals;
- Allegro joint states for fingertip kinematics and contact placement.

All of these signals are brought into a shared object-centric representation before they are used for SLAM or manipulation.

3.2 Dataset Variants

The GaussianFeels codebase uses three dataset variants: `feelsight`, `feelsight_real`, and `feelsight_occlusion`. They share the same high-level object/log layout, but they differ in how supervision is provided.

This follows the public FeelSight release structure, which separates the simulation, real-world, and occlusion-focused sequences into distinct dataset roots while keeping shared object assets alongside them.

Table 3.1: FeelSight variants used by GaussianFeels.

Dataset	Setting	Segmentation in pipeline	Main use
<code>feelsight</code>	simulation	GT object masks	base reconstruction and tracking
<code>feelsight_real</code>	real world	MobileSAM [20] masks	real-data evaluation
<code>feelsight_occlusion</code>	simulation	GT object masks	occlusion stress test

This split matters for the runtime. In simulation, the RealSense streams include object masks that can be used directly. In real-world data, those masks are not available, so the system builds the object crop through the MobileSAM path instead.

3.3 Problem Definition in the FeelSight Setting

In this setting, the system must solve three coupled problems at once.

1. **Maintain an object-centric surface model online.** The map must update frame by frame rather than through an offline batch reconstruction.
2. **Estimate object pose when supervision is removed.** Once the system operates in GT-init or SLAM mode, the object pose must be inferred from the evolving map and the incoming observations.
3. **Provide a useful object estimate to manipulation from the beginning of the episode.** A downstream policy cannot wait until the full surface has been explored physically.

These requirements motivate the architecture of GaussianFeels. The SLAM system addresses the first two problems, while the manipulation-side shape-completion module addresses the third.

3.4 Execution Modes

The same sequences can be processed in three pose-handling modes:

Table 3.2: Pose-handling modes used in GaussianFeels.

Mode	Frame 0	Frames 1+	Primary use
GT	ground truth	ground truth	reconstruction baseline
GT-init	ground truth	tracked	hybrid evaluation
SLAM	tracker seed	tracked	full online tracking

GT mode is useful for isolating reconstruction quality. GT-init separates frame-0 initialization from subsequent tracking. SLAM mode tests the whole system without object-pose supervision from the first frame.

3.5 Summary

The FeelSight setting gives GaussianFeels its problem structure: partial camera visibility, local tactile evidence, object-centric tracking, and a need for a useful object model from the beginning of manipulation. The next chapter asks how GaussianFeels turns that synchronized data stream into an online object-centric Gaussian reconstruction system.

Chapter 4

GaussianFeels SLAM

4.1 System Overview

This chapter answers the main method question of the thesis: how can synchronized manipulation data be turned into an object-centric Gaussian map that updates online under partial visibility? GaussianFeels addresses that question with a per-frame SLAM loop whose central state is the manipulated object itself rather than the surrounding scene.

At each frame, the runtime performs six conceptual stages:

1. ingest synchronized RGB-D, tactile, and hand-state observations;
2. build the current multimodal frame representation;
3. estimate or read the object pose;
4. synchronize and update the Gaussian map in the object frame;
5. render losses and optimize the active Gaussians;
6. export the updated reconstruction state and the payload needed by the manipulation branch.

When the manipulation module is enabled, a second process is started for the frame-0 shape bootstrap. In the intended deployment setup, that process runs on a separate GPU. The SLAM loop synchronizes with it after frame 0, before measured SLAM throughput begins.

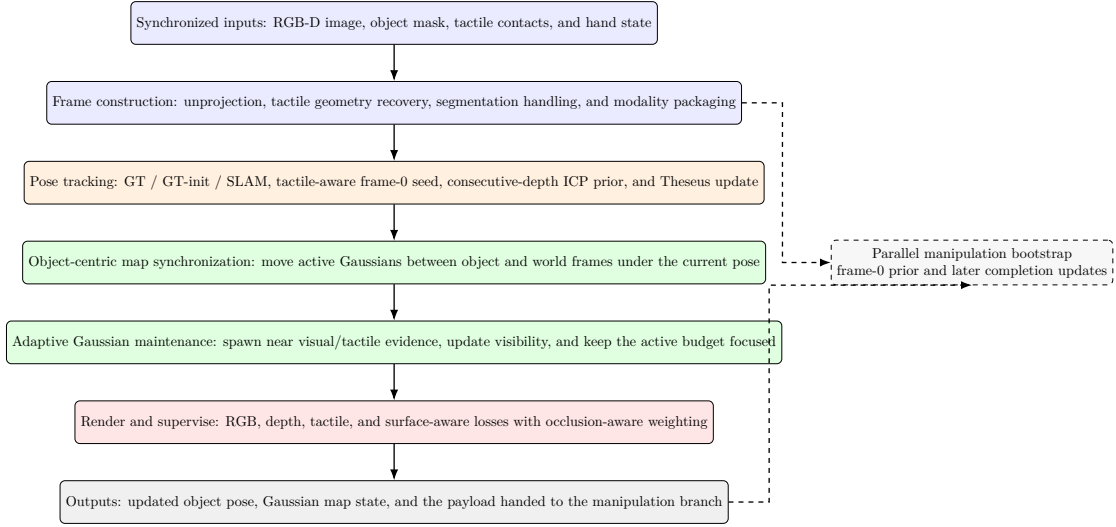


Figure 4.1: Per-frame data flow of GaussianFeels SLAM. Pose tracking sits in the middle of the loop because it is the transform that links the incoming observations to the object-centric Gaussian state.

Algorithm 1 Per-frame GaussianFeels SLAM update

Require: sensor frame $\mathcal{F}_t$, previous pose T_{WO}^{t-1} , object-frame Gaussian map $\mathcal{G}_{t-1}^O$

- 1: **if** $t = 0$ **then**
- 2: initialize T_{WO}^0 from GT pose or tactile-aware seed
- 3: **else**
- 4: estimate a consecutive-depth ICP prior
- 5: optimize only the current pose using frozen-map residuals and temporal priors
- 6: **end if**
- 7: synchronize the world-frame Gaussian state from $\mathcal{G}_{t-1}^O$ and T_{WO}^t
- 8: spawn and select Gaussians from camera and tactile evidence
- 9: render RGB, depth, tactile, and surface losses with occlusion-aware weights
- 10: optimize the active Gaussian parameters
- 11: write optimized Gaussians back into the object frame
- 12: export the updated pose and manipulation payload

4.2 Sensor Frame Construction

Each time step is represented as a synchronized multimodal frame. For each frame, the system prepares:

- RGB image and depth map from the selected RealSense camera;
- camera intrinsics and the camera-to-world transform used by the runtime;
- object pose when the sequence is evaluated in GT mode;
- segmentation masks from dataset ground truth in simulation or from a lightweight SAM variant (MobileSAM) in real-world sequences;
- tactile contact points and contact normals obtained from DIGIT images;
- hand state for fingertip pose recovery when needed.

The tactile pipeline converts each DIGIT image into a heightmap and then into contact geometry. For simulation, the background template is estimated from a set of low-variance frames. For real-world data, frame 0 is used as the conservative default background. The output of preprocessing is the synchronized frame

$$\mathcal{F}_t = \{I_t, D_t, M_t, K, T_{WC}^t, P_t^{\text{cam}}, P_t^{\text{tac}}, N_t^{\text{tac}}, h_t, T_{WO,GT}^t\}, \quad (4.1)$$

where I_t and D_t are the RGB and depth images, M_t is the object mask, K is the camera intrinsic matrix, T_{WC}^t is the world-to-camera pose,¹ P_t^{cam} is the camera point cloud, P_t^{tac} and N_t^{tac} are tactile contact points and normals, h_t denotes the hand state, and $T_{WO,GT}^t$ is present only when GT pose supervision is available.

This synchronized frame matters because every downstream stage operates on the same geometry packet. Reconstruction, tracking, and manipulation therefore all reason about the same time-aligned observation rather than about separately assembled modality streams.

¹The runtime stores T_{WC} in the world-to-camera (W2C) convention, so the camera position in world coordinates is $-R_{WC}^T t_{WC}$. This differs from the camera-to-world reading the WC subscript might otherwise suggest.

In real-world sequences, MobileSAM is invoked with `multimask_output=True` so that three mask candidates are returned per prompt. The candidate closest in area to a rolling reference — computed as the median of the last ten accepted mask areas — is selected, rather than the highest-IoU candidate. This area- based selection avoids two failure modes that arise from single-mask or score- ranked selection: small DIGIT sensor glow regions, which can score higher than the object under IoU ranking, and large background blobs that occasionally dominate the single-mask output. Negative prompts are also filtered: any negative prompt point within 60 pixels of a positive prompt is discarded, because close negative prompts were found to suppress correct object masks on rounded objects in pilot experiments.

4.3 Pose Tracking Inside the SLAM Loop

Pose tracking sits in the middle of the runtime because it links the current observations to the persistent Gaussian map. The same SLAM pipeline supports the GT, GT-init, and SLAM modes summarized in Table 3.2.

Frame-0 seeding. The tracker first establishes the initial object-frame reference. In GT and GT-init modes, frame 0 uses the provided pose directly. In SLAM mode, the translation and rotation seeds are estimated from the first visible depth cloud and optional tactile contacts.

The translation seed follows a priority-based selection. If a forward-kinematics grasp-center estimate is available, it is used directly. Otherwise, the seed is placed at the depth-cloud centroid offset by 2 cm in the view direction:

$$c_0 = \begin{cases} c_{\text{grasp}}, & \text{if FK grasp center available,} \\ c_{\text{depth}} + 0.02 \hat{v}, & \text{otherwise,} \end{cases} \quad (4.2)$$

where c_{grasp} is the FK-derived grasp center, c_{depth} is the centroid of the unprojected depth cloud, and $\hat{v}$ is the unit direction from the camera origin toward c_{depth} . The small view-direction offset compensates for the front-face bias of single-view depth.

The rotation seed enumerates all 24 axis-aligned orientations of the object’s PCA frame. Starting from the three principal eigenvectors v_1, v_2, v_3 of the centered depth cloud, every permutation and consistent-sign combination is evaluated, giving a discrete set $\mathcal{C}$ of candidates that covers all axis-aligned $\text{SO}(3)$ orientations without continuous fitting. Each candidate $R_k \in \mathcal{C}$ is assigned a scalar cost

$$\phi(R_k) = s_{\text{var}}(R_k) + 2 s_{\text{cam}}(R_k) + 0.5 s_{\text{up}}(R_k) + 10 s_{\text{vis}}(R_k) + 100 s_{\text{in}}(R_k), \quad (4.3)$$

and the rotation with the lowest cost is selected, $R_0 = \arg \min_{R_k \in \mathcal{C}} \phi(R_k)$. Here, s_{var} is the sum of depth-projection variances along the object x - and y -axes (favoring compact orientations), $s_{\text{cam}} = 1 - \langle R_k[:, 0], \hat{r}_{\text{cam}} \rangle$ penalizes deviation of the object x -axis from the camera right direction (breaking the dominant 180° in-plane PCA ambiguity), $s_{\text{up}} = 1 - |\langle R_k^\top e_z, e_z \rangle|$ rewards an upright object orientation, s_{vis} adds a large penalty when the camera viewpoint lies on the wrong side of the object in the candidate frame, and s_{in} is the mean inward-pointing penalty of tactile contact normals in the candidate frame, active only when tactile data is available. The large weight on s_{in} ensures that tactile normals dominantly constrain the final orientation whenever contact measurements exist.

Frozen-map signed-distance residuals. After seeding, tracking uses a frozen object map built from recent keyframe point clouds reprojected into the object frame. For a query point p , the map computes a differentiable signed distance from nearby anchor points q_i (Figure 4.3):

$$w_i(p) = \exp\left(-\frac{\|p - q_i\|_2^2}{2\sigma^2}\right), \quad (4.4)$$

$$\tilde{q}(p) = \frac{\sum_i w_i(p) q_i}{\sum_i w_i(p)}, \quad (4.5)$$

$$\tilde{n}(p) = \frac{\sum_i w_i(p) n_i}{\|\sum_i w_i(p) n_i\|_2}, \quad (4.6)$$

$$d_{\mathcal{M}}(p) = (p - \tilde{q}(p))^\top \tilde{n}(p), \quad (4.7)$$

Sign combinations for (v_1, v_2) ; v_3 fixed by right-hand rule

	$(+, +)$	$(+, -)$	$(-, +)$	$(-, -)$
$[v_1, v_2, v_3]$	R_1 chosen	R_2	R_3	R_4
$[v_1, v_3, v_2]$	R_5	R_6	R_7	R_8
$[v_2, v_1, v_3]$	R_9	R_{10}	R_{11}	R_{12}
$[v_2, v_3, v_1]$	R_{13}	R_{14}	R_{15}	R_{16}
$[v_3, v_1, v_2]$	R_{17}	R_{18}	R_{19}	R_{20}
$[v_3, v_2, v_1]$	R_{21}	R_{22}	R_{23}	R_{24}

Axis permutation

$R_0 = \arg \min_{R_k \in \mathcal{C}} \phi(R_k)$
 $\phi = s_{\text{var}} + 2 s_{\text{cam}} + 0.5 s_{\text{up}} + 10 s_{\text{vis}} + 100 s_{\text{in}}$

$|\mathcal{C}| = 24$ candidates covering all axis-aligned $\text{SO}(3)$ orientations ($3! \times 2^2 = 24$)

Figure 4.2: The 24-candidate PCA permutation search used for frame-0 rotation seeding. Each cell represents one axis-aligned $\text{SO}(3)$ candidate R_k , formed by choosing one of the six permutations of the three PCA eigenvectors (v_1, v_2, v_3) (rows) and one of the four consistent sign combinations (columns; the third axis sign is fixed by the right-hand rule). The highlighted cell is the selected candidate R_0 , which minimizes the five-term cost $\phi(R_k)$.

where n_i is the outward surface normal stored for anchor q_i . This gives a smooth Gaussian-density analogue of a point-cloud signed-distance field. The frozen map plays a conservative role: it changes only through accepted recent observations, so the tracker aligns against a stable geometric reference rather than against Gaussians that are being optimized at the same time.

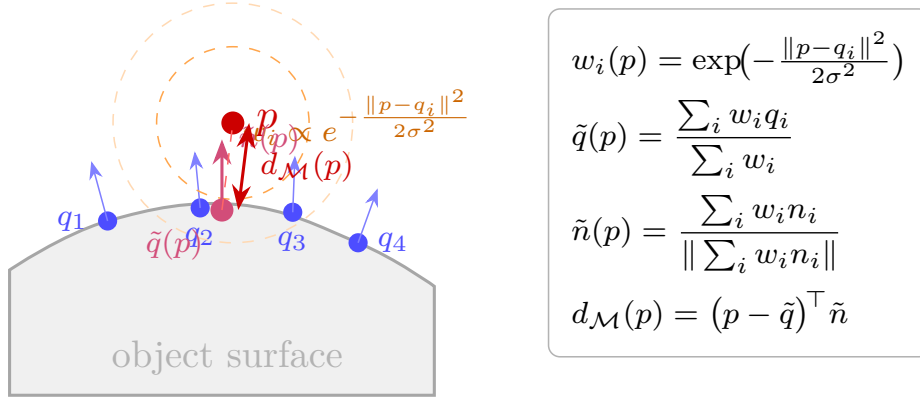


Figure 4.3: Illustration of the Gaussian-density signed-distance field used by the frozen object map. Blue dots are anchor points q_i sampled on the object surface with associated outward normals. For a query point p (red), Gaussian weights w_i fall off with distance (dashed circles). The weighted centroid $\tilde{q}(p)$ and blended normal $\tilde{n}(p)$ define a local tangent plane, and the signed distance $d_{\mathcal{M}}(p)$ is the projection of $p - \tilde{q}$ onto $\tilde{n}$.

Theseus-based pose optimization. For frames after the seed, the runtime samples object pixels from the current depth image, transforms those points into the world frame, and optimizes only the latest pose in the sliding window. Older window entries are kept fixed as constants. The optimization runs in two stages. The first stage minimizes the frozen-map residual for the latest pose alone. The second stage augments this with a temporal prior linking the latest pose to the penultimate window entry. The optimizer is implemented with the Theseus nonlinear optimization library [21].

When the active Gaussian count exceeds 50 splats, the tracker can optionally use a `_GaussianSplatSDF` backend in place of the point-cloud frozen map.² The results in Chapter 6 use the point-cloud frozen map unless otherwise noted. The active objective

²The `_GaussianSplatSDF` backend computes a soft-min Mahalanobis distance directly against the active Gaussian splats and provides a fully analytical SE(3) Jacobian. It activates automatically when `gaussian_sdf_min_splats = 50` is reached.

is

$$\begin{aligned}
& \min_{T_{WO}^t} \sum_{p \in P_t^{\text{cam}}} d_{\mathcal{M}}(T_{OW}^t p)^2 + \lambda_{\text{tac}} \sum_{p \in P_t^{\text{tac}}} d_{\mathcal{M}}(T_{OW}^t p)^2 \\
& + \lambda_{\text{tr}} \|t_t - t_{t-1}\|_2^2 + \lambda_{\text{rot}} \|\log(R_{t-1}^\top R_t)\|_2^2 \\
& + \lambda_{\text{icp},t} \|t_t - \hat{t}_t^{\text{icp}}\|_2^2 + \lambda_{\text{icp},r} \left\| \log\left(\hat{R}_t^{\text{icp}\top} R_t\right) \right\|_2^2, \tag{4.8}
\end{aligned}$$

where $T_{WO}^t = [R_t \ t_t]$ is the current object-to-world pose, T_{OW}^t is its inverse, P_t^{cam} and P_t^{tac} are the current camera and tactile observations in world coordinates, and $(\hat{R}_t^{\text{icp}}, \hat{t}_t^{\text{icp}})$ is the accepted ICP prior target.

Each term addresses a different failure mode. The camera residual anchors the pose to the currently visible surface, the tactile residual preserves contact consistency even when only a small region is seen, the temporal prior prevents frame-to-frame jitter, and the ICP prior broadens the convergence basin under larger motion. These coefficients are implementation hyperparameters chosen for stability rather than claimed as theoretically optimal.

Depth ICP initialization. Before nonlinear optimization, the tracker attempts ICP between the previous and current depth clouds in the world frame. The resulting transformation is accepted only when it passes four gates: inlier fitness ≥ 0.1 , RMSE ≤ 50 mm, rotation magnitude $\leq 30^\circ$, and translation magnitude ≤ 50 mm. All four gates must be satisfied; a failure on any one of them causes the tracker to fall back to the temporal prior alone. The fitness and RMSE gates filter geometrically weak alignments, while the rotation and translation gates discard ICP results that would imply implausibly large inter-frame motion.

4.4 Object-Centric Gaussian Map

Once the pose for the frame is available, the map is synchronized and updated in object coordinates. Each object is represented by anisotropic Gaussian primitives with object-frame mean, scale, rotation, opacity, color, and optional spherical-harmonic coefficients.

This is the representation that GaussianFeels renders, trains, exports, and passes to downstream modules.

The object frame is the source of truth. At the start of frame t , the current tracked pose T_{WO}^t is used to synchronize the Gaussians into the world frame for rendering:

$$\mu_i^W = R_{WO}^t \mu_i^O + t_{WO}^t, \quad (4.9)$$

$$q_i^W = q_{WO}^t \otimes q_i^O, \quad (4.10)$$

where μ_i^O and q_i^O are the object-frame Gaussian mean and quaternion, and $\otimes$ denotes quaternion multiplication. After the trainer updates the world-frame parameters, the system writes them back into the object frame:

$$\mu_i^O = R_{OW}^t \mu_i^W + t_{OW}^t, \quad (4.11)$$

$$q_i^O = q_{OW}^t \otimes q_i^W. \quad (4.12)$$

This bidirectional synchronization is what keeps the map object-centric even though supervision and rendering are evaluated in the world frame.

The Gaussian management stage is responsible for:

- enforcing the active Gaussian budget used during training;
- spawning new Gaussians from visual and tactile evidence;
- freezing inactive Gaussians so optimization focuses on the active subset;
- synchronizing object-frame and world-frame Gaussian state using the tracked object pose;
- exporting the Gaussian map to PLY when requested.

Two population-management mechanisms are especially important in the contact setting. First, the runtime can insert Gaussians directly at tactile contact points so that contact observations always have nearby primitives to constrain. Second, it can temporarily increase Gaussian density near tactile contacts so that locally important regions are not starved by the global active-budget limit. Together, these mechanisms

prevent the contact region from becoming under-represented during online optimization.

4.5 Online Training Objective

The online training loop is organized around four core supervision channels:

$$\mathcal{L}_{\text{core}} = \lambda_{\text{rgb}} \mathcal{L}_{\text{rgb}} + \lambda_{\text{depth}} \mathcal{L}_{\text{depth}} + \lambda_{\text{tactile}} \mathcal{L}_{\text{tactile}} + \lambda_{\text{surface}} \mathcal{L}_{\text{surface}}. \quad (4.13)$$

Here, $\mathcal{L}_{\text{rgb}}$ penalizes rendered color mismatch, $\mathcal{L}_{\text{depth}}$ aligns rendered and observed depth, $\mathcal{L}_{\text{tactile}}$ pulls the map toward contact geometry, and $\mathcal{L}_{\text{surface}}$ regularizes the Gaussians toward a coherent surface. The four coefficients determine how strongly the update trusts appearance, geometric alignment, contact geometry, and surface regularity.

In practice, the trainer does not trust those four weights as fixed constants throughout the sequence. When finger occlusion becomes significant, the runtime reduces the influence of visual losses and increases the influence of tactile losses. Let $\rho_{\text{occ}} \in [0, 1]$ denote the occlusion ratio produced from the dilated foreground-depth and Sobel-edge mask. The active weights are

$$s_{\text{vis}} = 1 - \rho_{\text{occ}}(1 - w_{\text{min}}), \quad (4.14)$$

$$\lambda'_{\text{rgb}} = s_{\text{vis}} \lambda_{\text{rgb}}, \quad (4.15)$$

$$\lambda'_{\text{depth}} = s_{\text{vis}} \lambda_{\text{depth}}, \quad (4.16)$$

$$\lambda'_{\text{tactile}} = (1 + \beta \rho_{\text{occ}}) \lambda_{\text{tactile}}, \quad (4.17)$$

where w_{min} is the minimum retained visual weight and β is the tactile boost factor.

This reweighting exists to prevent a predictable failure mode: under strong hand occlusion, RGB and depth residuals can become misleading while the tactile signal remains locally trustworthy. The parameters w_{min} and β are engineering hyperparameters chosen in pilot runs so that visual losses are never removed completely, but tactile super-

vision can dominate when the hand occupies a large fraction of the object view.

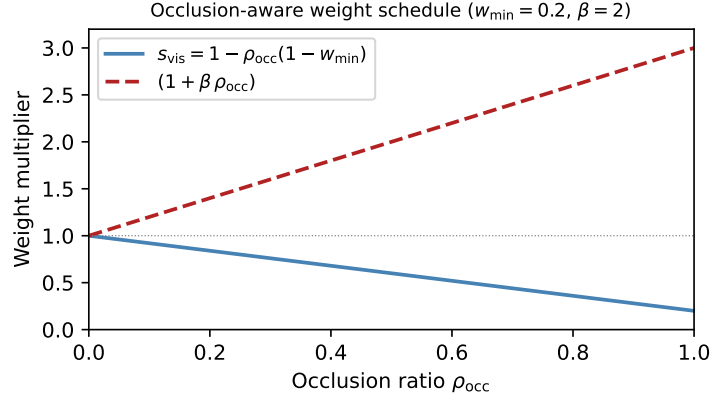


Figure 4.4: Occlusion-aware weight schedule. As the occlusion ratio ρ_{occ} increases, the visual weight multiplier s_{vis} decreases linearly from 1 to $w_{\min}$, while the tactile boost factor $(1 + \beta \rho_{occ})$ increases. The two curves cross at $\rho_{occ} = (1 - w_{\min})/\beta$, marking the transition where tactile supervision becomes stronger than the unoccluded visual baseline.

4.6 Runtime Coordination and Outputs

At each frame, the runtime coordinator performs the following actions:

1. receive the synchronized sensor data;
2. compute the pose estimate or read GT pose when appropriate;
3. synchronize the Gaussian map with that pose;
4. compute the training losses and update the map;
5. store outputs such as tracked poses, Gaussian maps, and optional PLY exports.

The same coordinator also hands the frame-0 payload and periodic updates to the parallel manipulation process described in Chapter 5.

4.7 Summary

GaussianFeels SLAM is an object-centric reconstruction runtime built around explicit Gaussian state, a pose tracker inside the update loop, and an online training objective

that combines visual and tactile supervision. This solves online accumulation and pose estimation, but it does not by itself provide a complete object estimate at the beginning of an episode. The next chapter explains the parallel manipulation module that addresses that early-episode completeness problem.

Chapter 5

Manipulation-Side Completion

5.1 Motivation

Chapter 4 solved online accumulation and pose estimation, but it left one important manipulation problem unresolved: at frame 0, the robot has seen only a small fraction of the object. A manipulation policy still needs an object estimate from the beginning of the episode. Starting from incomplete geometry, or from a generic placeholder object, reduces how well the policy can plan contact-rich motion. The manipulation module therefore adds an early shape estimate and then improves that estimate as real measurements arrive.

5.2 Parallel Runtime Structure

The manipulation module runs in parallel with GaussianFeels SLAM. In the intended setup, the SLAM process and the manipulation process use separate GPUs. The frame-0 shape estimate is started as soon as the frame-0 input is available, and the main runtime waits for that bootstrap step after frame 0 SLAM processing and before measured SLAM throughput begins.

In the implementation, this is not just a conceptual split. The manipulation branch is launched as a separate spawned process, communicates with the SLAM side through a bounded queue, and exposes an explicit `wait_for_frame0` barrier. That barrier is intentionally placed outside the reported SLAM FPS window so that the frame-0 image-to-3D bootstrap does not distort the online tracking throughput numbers.

The queueing policy is also asymmetric by design. The coordinator always keeps

the frame-0 bootstrap packet, while later frames may be dropped if the manipulation process falls behind. This matches the role of the module: frame 0 must produce the initial prior, whereas subsequent updates are best-effort refinements that should never stall SLAM.

The same coordinator supports both dataset playback and live integration with GaussianFeels. In dataset mode it loads frame-0 depth, RGB, segmentation, and poses from disk. In live mode it reconstructs the same frame-0 payload from the runtime sensor packet, extracts tactile contact points from the enabled fingers, and converts the pose convention used by GaussianFeels into the camera-to-world form expected by the manipulation alignment code.

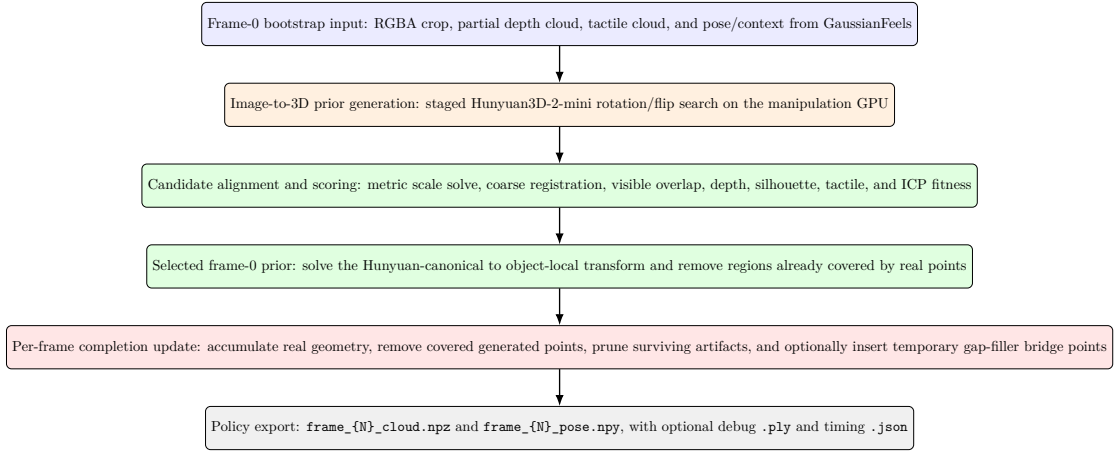


Figure 5.1: Manipulation-side pipeline. The frame-0 bootstrap produces an initial object estimate, and later updates progressively replace generated geometry with measured geometry.

Conceptually, the manipulation branch has three responsibilities:

- build the frame-0 prior;
- update that prior with real measurements over time;
- export the best available object estimate to the policy.

5.3 Frame-0 Prior with Hunyuan3D-2-mini

The frame-0 prior is produced with Hunyuan3D-2-mini, a lightweight single-image image-to-3D generator [3], [19]. At a high level, the model takes an RGBA crop of the

object and produces a 3D shape proposal that can be sampled into a point cloud. In this thesis, the important part is not generic asset generation. It is the use of image-to-3D generation as an early object prior for manipulation. Chapter 6 reports the frame-0 prior benchmark used to choose Hunyuan3D-2-mini for the deployed manipulation pipeline.

GaussianFeels does not use the raw model output directly. The deployed method wraps the pretrained generator with several additional steps:

- build an exact RGBA crop from the segmentation mask;
- generate a staged bank of rotated and mirrored image variants to handle the model’s roll sensitivity;
- run those variants through a persistent worker on the manipulation GPU;
- score each candidate against the real frame-0 partial cloud;
- solve the constant Hunyuan-canonical to object-local transform;
- keep the best candidate and merge it with measured geometry.

The contribution in this chapter lies in that integration around the pretrained model rather than in modifications to the base Hunyuan generator itself.

The orientation search is adaptive rather than exhaustive. The implementation starts from the four cardinal rotations. If the best cardinal candidate is not already decisive, it evaluates left-right flipped cardinals. Diagonal variants are considered only after that stage, and even then the search may remain focused around the strongest cardinal branches instead of evaluating every possible diagonal branch. This reduces unnecessary Hunyuan calls while still covering the model’s most important failure mode, namely sensitivity to object roll in the input crop.

For each generated branch, the frame-0 registration stage solves a constant Hunyuan-canonical to object-local similarity transform

$$x_O = s R(x_H - c_H) + t, \tag{5.1}$$

where x_H is a point in the Hunyuan canonical frame, c_H is the canonical center, s is the scale, R is the rotation, and t is the object-local translation.

The solved transform is not obtained from a single ICP call. The implementation first estimates metric scale from the visible frame-0 depth cloud, using the two object extents most perpendicular to the camera direction so the estimate is driven by the observed silhouette rather than the largely unobserved depth axis. It then evaluates two coarse registration families: a PCA-based pose enumeration and an FPFH-based registration path with additional fallback coarse-search candidates when the first coarse result is weak. Those candidates are refined against the visible frame-0 surface, and the winning branch is then polished further by local searches over scale, rigid pose, rotation, and translation. In effect, the deployed frame-0 registration is a small search pipeline rather than a single rigid alignment primitive.

Candidate branches are scored against the frame-0 partial observation. When tactile points are available, the observation score is

$$S_{\text{obs}} = 0.35 F_1^{\text{vis}} + 0.25 S_{\text{depth}} + 0.15 \text{IoU}_{\text{sil}} + 0.25 S_{\text{tac}}, \quad (5.2)$$

and otherwise

$$S_{\text{obs}} = 0.50 F_1^{\text{vis}} + 0.35 S_{\text{depth}} + 0.15 \text{IoU}_{\text{sil}}. \quad (5.3)$$

The final branch-selection score also includes the registration fitness:

$$S_{\text{final}} = 0.92 S_{\text{obs}} + 0.08 F_{\text{icp}}. \quad (5.4)$$

Here, F_1^{vis} measures visible-surface overlap between the aligned generated branch and the frame-0 partial cloud, S_{depth} measures rendered depth agreement, IoU_{sil} measures silhouette consistency, S_{tac} measures tactile overlap, and F_{icp} is the inlier fitness of the final registration step. These weights are implementation heuristics. Visible overlap carries the largest share because it best measures whether the candidate explains the observed partial shape, tactile overlap is weighted strongly when available because it anchors contact geometry, and ICP receives only 8% so that it acts as a geometric

tie-breaker rather than overwhelming the observation score.

The observation terms are all evaluated in the deployed object frame. F_1^{vis} is computed on the visible generated surface, not on the full hallucinated shape, because the frame-0 real observation constrains only the currently visible portion of the object. The depth term compares rendered and observed depth in image space, the silhouette term checks whether the generated object occupies the correct image region, and the tactile term adds a contact-geometry constraint when finger measurements are available. Together, these terms prevent a branch from winning solely because it is globally close in point-cloud space while being inconsistent with the actual frame-0 image and touch evidence.

Algorithm 2 Frame-0 manipulation bootstrap

Require: frame-0 RGBA crop, segmentation mask, partial real cloud, tactile cloud if available

- 1: build an exact object crop from the mask
- 2: evaluate cardinal rotations first
- 3: **if** the best cardinal branch is not decisive **then**
- 4: evaluate mirrored cardinal branches
- 5: expand to diagonal variants, either fully or around the strongest candidates
- 6: **end if**
- 7: run Hunyuan3D-2-mini on each required batch through the persistent worker
- 8: **for all** generated branches produced by that staged search **do**
- 9: solve the Hunyuan-canonical to object-local transform
- 10: score the aligned branch with visible-surface, depth, silhouette, tactile, and ICP metrics
- 11: **end for**
- 12: keep the highest-scoring aligned branch
- 13: polish the winning transform by local scale and pose refinement
- 14: remove generated regions already covered by frame-0 real points
- 15: export the combined point cloud and frame-0 pose

5.4 Progressive Replacement by Real Geometry

After frame 0, the module no longer treats the generated shape as fixed. It maintains separate sets of generated points and measured points. Whenever newly accumulated real observations cover a region that had previously been supplied by the generative prior, the generated points in that region are removed.

This design has two consequences:

- the shape prior is useful immediately, before the robot has explored the whole surface;
- the object model becomes increasingly measurement-driven as the sequence progresses.

Let P_t^{gen} denote the surviving generated points, P_t^{real} the accumulated real points, and ΔP_t^{real} the new real points arriving at update t . The shape updater applies

$$P_{t+1}^{\text{real}} = P_t^{\text{real}} \cup \{p \in \Delta P_t^{\text{real}} \mid \text{dist}(p, P_t^{\text{real}}) \geq \tau_{\text{merge}}\}, \quad (5.5)$$

$$P_{t+1}^{\text{gen}} = \{p \in P_t^{\text{gen}} \mid \text{dist}(p, P_{t+1}^{\text{real}}) \geq \tau_{\text{cover}}\}, \quad (5.6)$$

with $\tau_{\text{merge}} = 3 \text{ mm}$ and $\tau_{\text{cover}} = 5 \text{ mm}$ in the current implementation. To bound memory and runtime, the accumulated real set is capped at $|P_t^{\text{real}}| \leq 10^5$ points; when this limit is reached, the combined real set is randomly subsampled to the cap before the coverage step.

The asymmetry between the two thresholds is deliberate. The smaller merge threshold keeps the accumulated real cloud conservative by avoiding duplicate measured points, while the larger cover threshold removes generated geometry once real data explains the same region with a clear margin. These values are engineering thresholds chosen to favor stable replacement rather than fast churn.

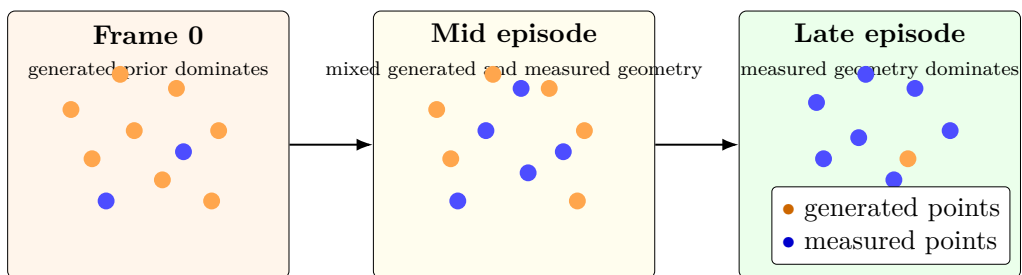


Figure 5.2: Schematic of progressive replacement. Orange points are generated prior geometry and blue points are measured geometry. As the episode progresses, measured observations cover and remove generated regions, so the object model becomes increasingly measurement-driven.

The implementation also performs a lightweight outlier pruning step on the surviving generated set after coverage-based removal. This pruning is deliberately asymmetric:

it is applied only to generated points, not to the accumulated real points, because the real points are treated as trusted measurements while the generated points remain hypotheses that may still contain obvious isolated artifacts.

In addition, the runtime can insert a temporary bridge cloud between the real and generated surfaces when those two sets drift apart spatially. The bridge is computed from boundary points on the measured and generated clouds and fills the gap with short interpolated segments. Its role is not to redefine the exported object surface; it is an internal stabilization aid for cases where measured and generated patches would otherwise leave a visible seam through the object.

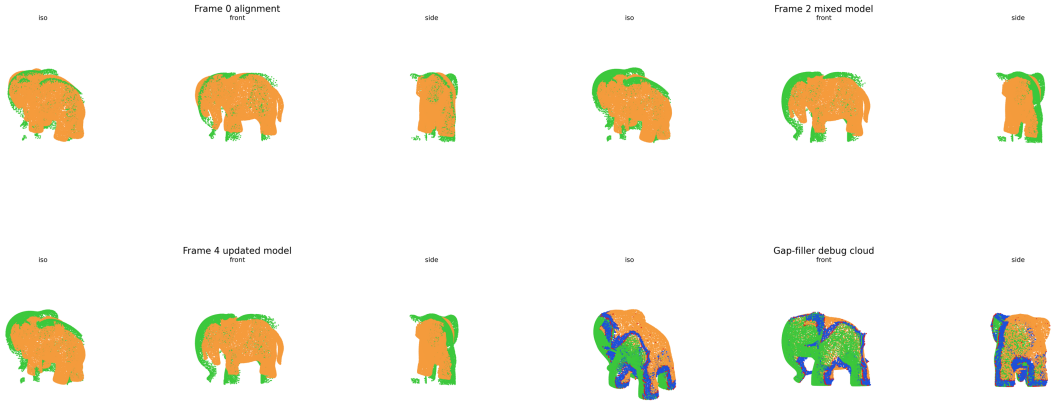


Figure 5.3: Real manipulation-side progression from the saved five-frame sweep and the gap-filler preview. In the first three panels, orange points come from the frame-0 generative prior and green points come from measured geometry accumulated during manipulation. By frame 4, more of the object surface has been replaced by measurements. The final panel shows the gap-filler debug cloud, where blue points are temporary bridge points inserted only as an internal seam-stabilization aid; they are not part of the policy-facing export.

5.5 Policy-Facing Output

The manipulation module exports the object estimate in a form that a downstream policy can consume directly: a per-frame compressed NumPy cloud archive `frame_{N}_cloud.npz` together with an object-pose file `frame_{N}_pose.npy`. The cloud archive stores a structured point cloud whose points are labeled by provenance, so the policy can distinguish surfaces that have been measured from surfaces that are still generated

priors.

Concretely, the policy-facing export writes a compressed point-cloud archive and an object pose for each frame. The cloud archive stores the combined point set, the provenance labels, and the explicit real/generated subsets, while the pose file stores the object-to-world transform. Optional debug artifacts include a provenance-colored PLY for inspection and a timing JSON for metrics/debugging. The bridge points discussed above are kept as an internal runtime aid and are not part of the production policy contract.

5.6 Summary

The manipulation module is an addition to GaussianFeels SLAM, not a replacement for it. Chapter 4 solved online reconstruction and tracking; this chapter solved early-episode completeness. The deployed method uses Hunyuan3D-2-mini to produce a frame-0 prior, searches over orientation variants, solves a frame-0 object-frame transform with image, depth, and tactile scoring, and then lets real geometry take over as exploration proceeds. The runtime also supports live GaussianFeels integration, provenance-aware export, and internal gap filling for unstable seams between real and generated geometry. The final comparative evaluation of frame-0 priors is reserved for Chapter 6; this chapter focuses only on the deployed image-to-3D path and the real-data replacement mechanism.

Chapter 6

Experiments

This chapter defines the final experimental protocol and reserves the figures and tables where the remaining quantitative results will be inserted. The frame-0 shape-prior comparison has already been completed and is reported here.

6.1 Evaluation Questions

The experiments in this thesis are organized around five questions:

1. How accurately does GaussianFeels reconstruct object geometry in the FeelSight setting?
2. How accurately does the pose tracker operate in GT, GT-init, and SLAM modes?
3. How robust is the system under strong occlusion and other hard visual conditions?
4. Which frame-0 shape prior is best for the manipulation module?
5. How much does a better initial shape estimate help downstream manipulation?

6.2 Evaluation Metrics

The final experiments will report:

- reconstruction metrics such as Chamfer distance and F-score;
- pose-tracking metrics such as translation and rotation error;
- runtime metrics such as latency, FPS, and GPU memory;

- manipulation-side metrics such as policy utility or task success when those experiments are completed.

For shape-completion model selection, F-score at 5 mm is the primary accuracy metric, together with runtime and output density. In that benchmark, each saved point cloud is first aligned with the same frame-0 registration pipeline used by the deployed manipulation module and is then evaluated against the ground-truth object surface in metric object coordinates.

6.3 Reserved Result Figures



Figure 6.1: Reserved qualitative reconstruction comparison across the three evaluation settings.

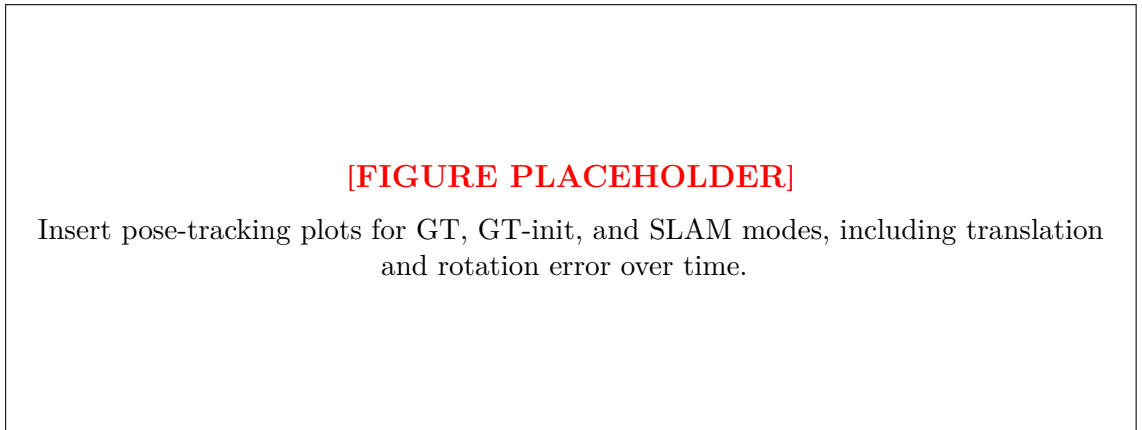


Figure 6.2: Reserved pose-tracking plots.

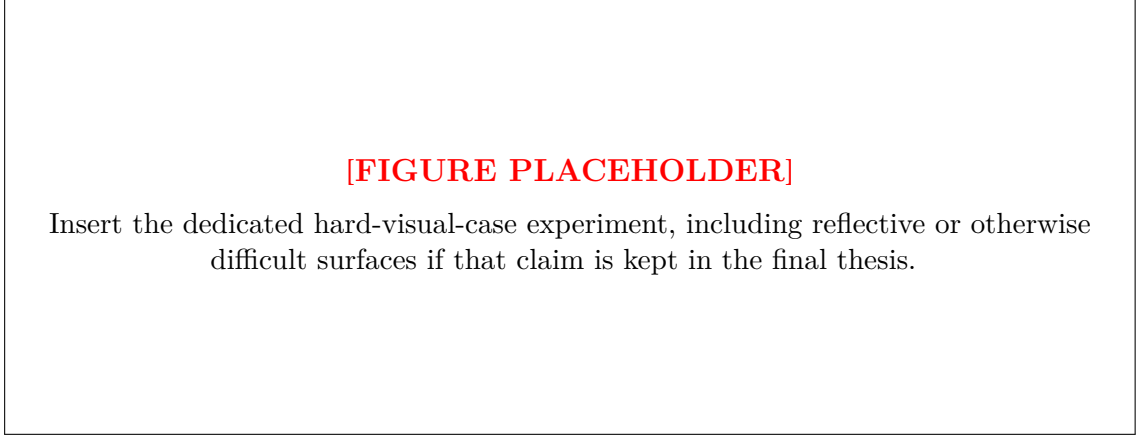


Figure 6.3: Reserved figure for hard-visual-condition experiments.

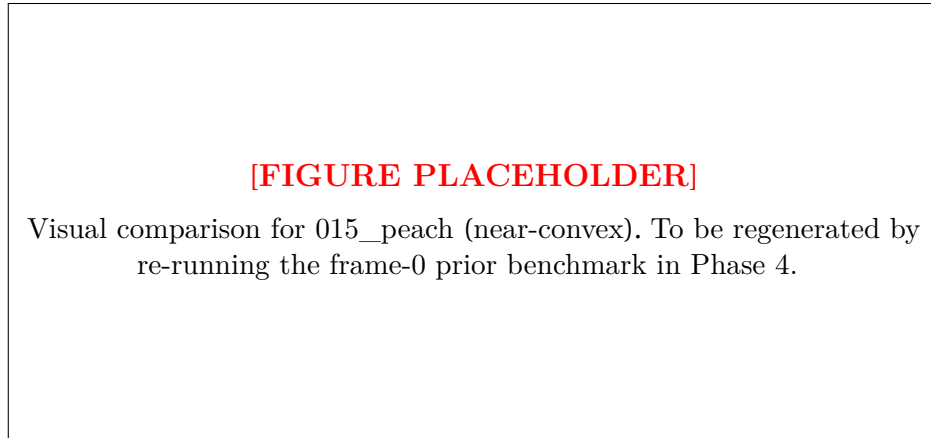


Figure 6.4: High-resolution qualitative comparison for a near-convex object trial. Each sheet shows the ground-truth point cloud and the saved model outputs from four fixed viewpoints after normalization for shape-only visual comparison. The row labels include the aligned F-score at 5 mm for that trial. On this peach example, Hunyuan3D-2-mini and Fast-SAM3D recover the object category most faithfully, while Geco and RGB2Point visibly collapse.

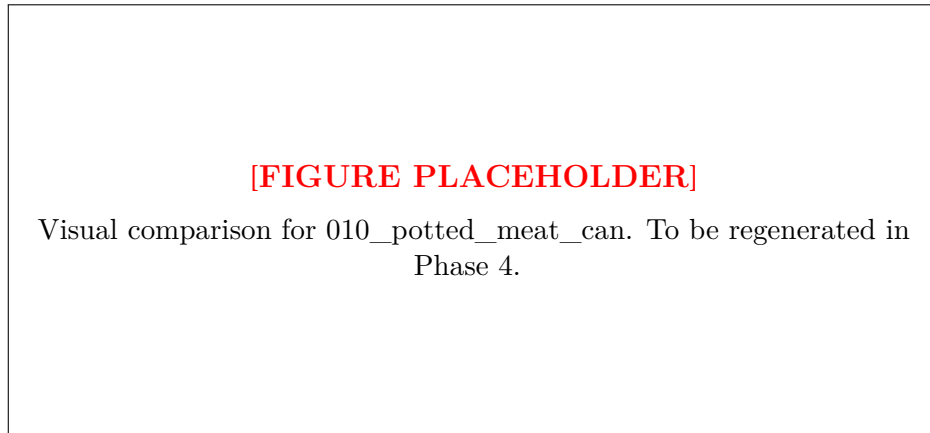


Figure 6.5: Qualitative comparison for a box-like food container. On this trial, Hunyuan3D-2-mini preserves the elongated can geometry more reliably than the other methods, while several alternatives either over-round the shape or collapse toward a generic support volume.

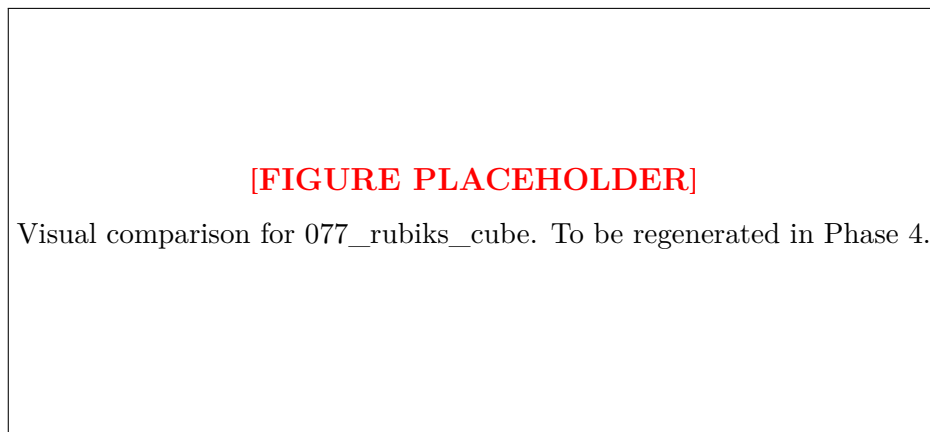


Figure 6.6: Qualitative comparison for a structured cuboid object. This example shows why the benchmark cannot be interpreted from F-score alone: TripoSR reaches a higher score than Hunyuan3D-2-mini on this trial but still rounds the object into a generic blob instead of preserving a cuboid shape.

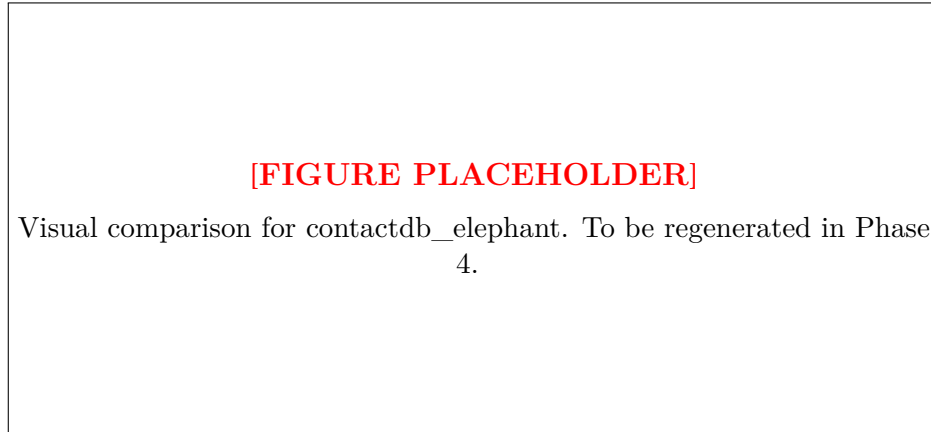


Figure 6.7: Qualitative comparison for an articulated animal-shaped object. Several methods achieve moderate scores despite losing the leg and trunk structure that would matter for manipulation-aware reasoning.

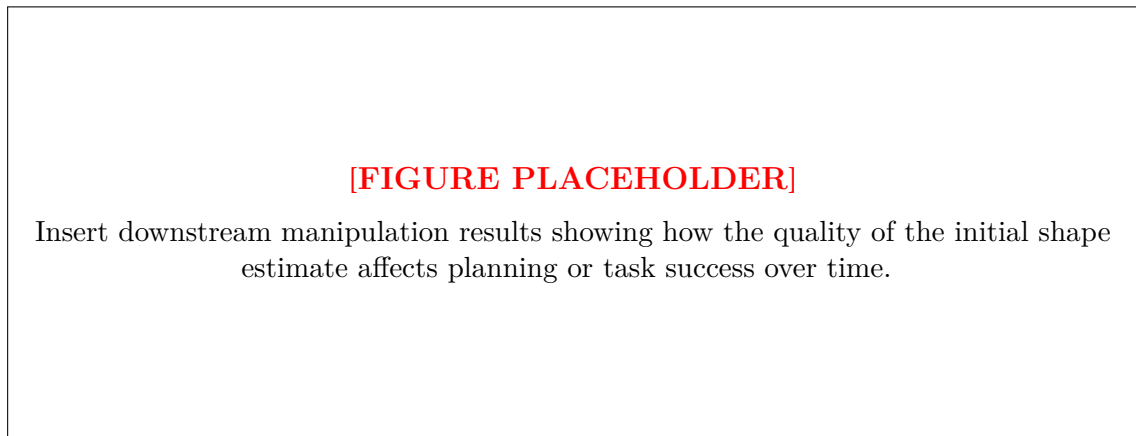


Figure 6.8: Reserved figure for the effect of shape priors on manipulation.

6.4 Reserved Result Tables

6.5 Frame-0 Prior Selection Results

The frame-0 prior benchmark was run on eight objects with five trials each, giving 40 object-trials in total. Each trial reused the saved point clouds and runtime artifacts from the image-to-3D benchmark and recomputed the final evaluation metric with the deployed frame-0 alignment pipeline. This produced a single manipulation-facing F-score at 5 mm for each saved prior. Two Fast-SAM3D runs and four RGB2Point runs failed.

Benchmark protocol. The benchmark evaluates the object prior that is actually handed to the manipulation stack at frame 0. For each saved model output, the point cloud is first aligned with the same production registration routine described in Chapter 5. The aligned cloud is then compared against the ground-truth object surface in object coordinates. This matters because the manipulation module never uses a canonical image-to-3D prediction directly. It uses the prediction only after it has been scaled, rotated, and translated into the object frame.

Why these metrics matter. The benchmark reports runtime, success count, F-score at 5 mm, Chamfer distance, precision, and recall. F-score captures thresholded surface agreement within the tolerance used by the manipulation-side evaluation. Chamfer distance measures the average geometric discrepancy over the full aligned surface and is more sensitive to broad geometric drift than a thresholded hit rate alone. Precision captures over-coverage: a method can hurt precision by hallucinating extra surface that does not belong to the object. Recall captures under-coverage: a method can hurt recall by missing real object regions. Runtime and success count matter because the prior is generated online at frame 0 and must be available reliably enough to support the rest of the manipulation episode.

Table 6.3 shows the resulting model ranking. Hunyuan3D-2-mini achieved the best mean F-score and therefore became the default frame-0 prior in GaussianFeels. It also

achieved the lowest average Chamfer distance while maintaining a full 40/40 success rate. That combination is important: Hunyuan3D-2-mini was not chosen because it was the fastest model, but because it gave the most favorable overall balance between accuracy, reliability, and object-specific geometric fidelity.

Fast-SAM3D remained a strong baseline. Its average F-score is close to InstantMesh and not far behind Hunyuan3D-2-mini, and its recall is the highest among the tested models. However, it is the slowest method in the benchmark and the archived sweep already contained two missing runs, which makes it less attractive as the deployed frame-0 prior.

InstantMesh is the main speed-quality trade-off baseline. It runs much faster than Hunyuan3D-2-mini and reaches a competitive average F-score and Chamfer distance. However, the qualitative comparisons show that it often rounds away object-defining structure, especially on constrained shapes such as the Rubik’s cube. For manipulation, that distinction matters: a generic support shape can score reasonably while still failing to preserve the geometry that influences contact planning or action selection.

TripoSR is the weaker but still relevant fourth competitor in this benchmark. It can obtain moderate F-scores on some near-convex objects because F-score rewards nearest-neighbor surface coverage within a fixed tolerance. It does not strongly penalize rounded category-level approximations, missing protrusions, or part-topology errors once enough of the generated surface lands within the 5 mm threshold after alignment. The qualitative comparison therefore matters for model selection: it reveals whether a method is recovering object-specific geometry or merely matching a coarse support shape well enough to score numerically.

Geco and RGB2Point were not competitive in the final decision. Across the saved visual comparisons, both methods tend to collapse toward generic low-fidelity point sets that change little across objects and trials. They remain in the table for completeness, but the meaningful deployment comparison is between Hunyuan3D-2-mini, Fast-SAM3D, InstantMesh, and TripoSR.

Deployment decision. Hunyuan3D-2-mini is the deployed frame-0 prior. Among the tested models, it provides the best average geometric accuracy by both F-score and Chamfer while also preserving the most faithful object identity across the representative visual comparisons. Fast-SAM3D is competitive and remains an informative baseline, but its runtime and missing saved runs make it less robust as the default online prior. InstantMesh is attractive on runtime alone, but the observed loss of object-specific structure weakens it as a manipulation-facing initialization. The remaining models are either too sparse, too generic, or too geometrically weak to justify deployment as the frame-0 prior.

Table 6.1: Reserved table for GaussianFeels reconstruction quality.

Dataset	Mode	Chamfer	F-score@5mm	Runtime note
feelsight	GT	TBD	TBD	TBD
feelsight	GT-init	TBD	TBD	TBD
feelsight	SLAM	TBD	TBD	TBD
feelsight_ real	GT-init/ SLAM	TBD	TBD	TBD
feelsight_ occlusion	GT-init/ SLAM	TBD	TBD	TBD

Table 6.2: Reserved table for pose-tracking accuracy.

Dataset	Mode	Translation error	Rotation error
feelsight	GT-init	TBD	TBD
feelsight	SLAM	TBD	TBD
feelsight_ real	GT-init/ SLAM	TBD	TBD
feelsight_ occlusion	GT-init/ SLAM	TBD	TBD

Table 6.3: Frame-0 shape-prior comparison on the saved 40-trial benchmark.

Model	Time (s)	F@5mm	Chamfer (mm)	Precision	Recall	Success	Decision
Hunyuan3D-2-mini	28.36	0.689	10.03	0.715	0.676	40 / 40	chosen prior
Fast-SAM3D	35.68	0.633	12.03	0.612	0.714	38 / 40	strong baseline
InstantMesh	15.26	0.639	11.01	0.649	0.646	40 / 40	faster, less faithful
TripoSR	9.86	0.576	12.57	0.603	0.564	40 / 40	weaker geometry
RGB2Point	5.30	0.484	16.79	0.583	0.427	36 / 40	sparse, unstable
Geco	14.20	0.400	22.11	0.324	0.536	40 / 40	generic shapes

Table 6.4: Reserved runtime table for the final SLAM system.

Dataset	FPS	Mean latency	GPU memory
feelsight	TBD	TBD	TBD
feelsight_real	TBD	TBD	TBD
feelsight_occlusion	TBD	TBD	TBD

Table 6.5: Reserved table for the effect of shape priors on downstream manipulation.

Initial prior	Task metric	Early-episode performance	Final performance
Hunyuan3D-2-mini	TBD	TBD	TBD
TripoSR	TBD	TBD	TBD
InstantMesh	TBD	TBD	TBD
Geco	TBD	TBD	TBD
RGB2Point	TBD	TBD	TBD
Fast-SAM3D	TBD	TBD	TBD

6.6 Reserved Notes

PLACEHOLDER

Insert the remaining SLAM, tracking, hard-visual-case, and downstream manipulation results after those experiment sweeps are complete. In particular, any claim about reflective surfaces or other hard visual cases should only appear in the final thesis if it is supported by a dedicated experiment reported in this chapter.

Chapter 7

Conclusion

7.1 What This Thesis Establishes

This thesis shows that explicit object-centric Gaussian maps can unify three requirements that are usually treated separately in manipulation systems: online multimodal reconstruction, object pose tracking under partial visibility, and immediate delivery of a usable object model to downstream manipulation. In GaussianFeels, the same state is optimized from synchronized camera and tactile evidence, transformed by the tracked object pose, and handed to a parallel manipulation branch without an intermediate surface-extraction step.

The broader implication is not limited to one dataset. Whenever manipulation produces synchronized RGB-D observations, tactile contact geometry, and hand state under heavy self-occlusion, the core perception problem is not merely to recover a final surface after the episode. It is to maintain an object state that remains usable throughout the episode while the robot is still acting. This thesis argues that explicit Gaussian representations matter in that regime because they are fast to render, direct to transform, and naturally compatible with manipulation-facing outputs.

7.2 Limitations

Several limitations bound the scope of what this thesis establishes.

Short-episode single-object setting. GaussianFeels is designed around short, single-object manipulation sequences of the kind present in FeelSight. The runtime does not

include a place-recognition mechanism or loop-closure step, so drift that accumulates over long episodes cannot be corrected retrospectively. The results reported here should therefore be interpreted in that bounded setting rather than as evidence for general long-horizon SLAM.

Frame-0 seeding robustness. The frame-0 translation and rotation seed is estimated from the depth cloud available at the first frame. When the visible depth cloud is sparse, noisy, or badly segmented, the 24-candidate PCA search may select a sub-optimal seed. In that case, the Theseus optimizer is responsible for recovering toward a correct pose, but a large initial error increases the risk of convergence to a local minimum. The GT-init mode avoids this risk entirely; results in SLAM mode therefore also reflect the quality of the frame-0 seed.

Hunyuan3D-2-mini orientation sensitivity. The frame-0 image-to-3D prior depends on Hunyuan3D-2-mini, which is sensitive to the roll of the input crop. The staged orientation search in the manipulation module mitigates this, but the search is not exhaustive and can still miss the best candidate on highly symmetric or textureless objects. The benchmark in Chapter 6 reports success rates and F-scores that reflect this variability.

Dual-GPU dependency. The intended deployment assigns SLAM to one GPU and the manipulation branch to a second GPU. On a single-GPU system, the parallel process must share the device with the SLAM trainer, which affects the frame-0 bootstrap time and may reduce SLAM throughput while the Hunyuan3D inference is running.

Hyperparameter sensitivity. The occlusion-aware loss weights ($w_{\min}, \beta$), the ICP acceptance thresholds, and the real-point accumulation cap (10^5) are engineering heuristics set in pilot runs. The system has not been evaluated under a systematic sweep of those parameters, and different objects or environments may benefit from different settings.

7.3 Future Work

Several directions extend the work in this thesis.

Learned occlusion and weight adaptation. The occlusion-aware weight schedule is currently driven by a hand-tuned geometric mask. Replacing the manual threshold with a learned occlusion predictor, or jointly optimizing the loss weights end-to-end, could improve robustness across objects and lighting conditions without requiring per-dataset pilot tuning.

Tighter reconstruction-to-manipulation integration. The current interface between the SLAM loop and the manipulation branch is one-directional: the SLAM side provides frame-0 data and periodic point-cloud updates, while the manipulation branch exports a policy-facing object model. A bidirectional interface where the manipulation prior feeds uncertainty information back into the Gaussian spawning or loss weighting could make the system more robust in the early frames before enough real geometry has accumulated.

Multi-object and articulated scenes. GaussianFeels currently tracks one object per sequence. Extending to multiple simultaneously manipulated objects, or to articulated structures with moving parts, would require managing multiple Gaussian maps and associating incoming observations with the correct object at each frame.

Longer episodes and global consistency. For episodes long enough to accumulate significant drift, adding a place-recognition module or a sparse global pose graph would allow the tracker to correct earlier pose estimates and maintain a globally consistent object map. This is a natural next step once the per-frame alignment quality is validated on the short-episode benchmark.

Faster frame-0 priors. InstantMesh ran significantly faster than Hunyuan3D-2-mini in the prior benchmark while maintaining competitive accuracy. A deployment that requires sub-10-second frame-0 bootstraps, or a setting where compute is more

constrained, could benefit from using InstantMesh as the default prior, possibly with a post-processing step to recover object-specific structure that the faster model tends to lose.

Policy integration experiments. The downstream manipulation evaluation in Chapter 6 was scoped to shape-prior selection rather than to a full policy execution experiment. Future work should connect the provenance-labeled point-cloud export directly to a contact-rich manipulation policy and measure how much the quality of the frame-0 prior, and its progressive replacement by real geometry, affects actual task success over a manipulation episode.

References

- [1] S. Suresh, H. Qi, T. Wu, *et al.*, “Neuralfeels with neural fields: Visuotactile perception for in-hand manipulation,” *Science Robotics*, adl0628, 2024.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” in *ACM SIGGRAPH*, 2023.
- [3] Tencent Hunyuan Team, *Hunyuan3D-2: Scaling diffusion models for high resolution textured 3D assets generation*, Technical Report, 2025. arXiv: [2501.12202](#).
- [4] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *ECCV*, 2020.
- [5] J. Zhao, M. Bauzá, and E. H. Adelson, “Fingerslam: Closed-loop unknown object localization and reconstruction from visuo-tactile feedback,” in *ICRA*, 2023, pp. 8033–8039.
- [6] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “SplatAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM,” in *CVPR*, 2024.
- [7] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian splatting SLAM,” in *CVPR*, 2024.
- [8] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, *Gaussian-SLAM: Photo-realistic dense SLAM with gaussian splatting*, 2023. arXiv: [2312.10070](#).
- [9] K. R. Barad, A. Richard, J. Dentler, M. Olivares-Mendez, and C. Martinez, *Object-centric reconstruction and tracking of dynamic unknown objects using 3D Gaussian splatting*, 2024. arXiv: [2405.20104](#).
- [10] Y. Li and D. Pathak, *Object-aware gaussian splatting for robotic manipulation*, ICRA 2024 Workshop on 3D Manipulation, 2024.
- [11] W. Yuan, S. Dong, and E. H. Adelson, “GelSight: High-resolution robot tactile sensors for estimating geometry and force,” *Sensors*, 2017.

-
- [12] M. Lambeta, P.-W. Chou, S. Tian, *et al.*, “DIGIT: A novel design for a low-cost compact high-resolution tactile sensor with application to in-hand manipulation,” *IEEE Robotics and Automation Letters*, 2020.
 - [13] S. Wang, M. Lambeta, P.-W. Chou, and R. Calandra, “GelSight Wedge: Measuring high-resolution 3D contact geometry with a compact robot finger,” in *ICRA*, 2021.
 - [14] S. Wang, M. Lambeta, P.-W. Chou, and R. Calandra, “TACTO: A fast, flexible, and open-source simulator for high-resolution vision-based tactile sensors,” *IEEE Robotics and Automation Letters*, 2022.
 - [15] A. Swann, M. Strong, W. K. Do, G. Sznaier Camps, M. Schwager, and M. I. Kennedy, “Touch-GS: Visual-tactile supervised 3D Gaussian splatting,” in *IROS*, 2024.
 - [16] M. Comi, A. Tonioni, J. Tremblay, *et al.*, “Snap-it, tap-it, splat-it: Tactile-informed 3D Gaussian splatting for reconstructing challenging surfaces,” in *2025 International Conference on 3D Vision (3DV)*, 2025, pp. 1134–1143.
 - [17] Y. Chen and G. Medioni, “Object modelling by registration of multiple range images,” *Image and Vision Computing*, 1992.
 - [18] A. Segal, D. Haehnel, and S. Thrun, “Generalized-ICP,” in *RSS*, 2009.
 - [19] Tencent, *tencent/Hunyuan3D-2mini*, Hugging Face model card, Accessed 2026-04-18, 2025.
 - [20] C. Zhang, D. Han, Y. Qiao, *et al.*, “Faster segment anything: Towards lightweight SAM for mobile applications,” *arXiv:2306.14289*, 2023.
 - [21] L. Pineda, T. Fan, M. Monge, *et al.*, “Theseus: A library for differentiable non-linear optimization,” in *NeurIPS*, 2022.

Appendix A

Default Runtime Configuration

A.1 Baseline Optuna Configuration

The default `feelsight` baseline uses the tuned core parameters from the bundled baseline configuration file.

Table A.1: Default baseline configuration loaded automatically for `feelsight` datasets.

Category	Parameter	Value
Training	learning rate	0.004705650404061025
	optimizer	sparse_adam
	weight decay	1.4668295922342405e-05
	lr decay	0.9960010943893057
	lr reduction cooldown	5 frames
Loss weights	render RGB	0.5024611257495276
	render depth	0.24411485271580183
	tactile	0.9589686161643609
	surface constraints	0.5807643614893898

Appendix B

Multi-Object Optimization Report

The repository also contains a broader optimization report for an 8-object split over multiple FeelSight sequences.

Table B.1: Summary of the multi-object optimization report bundled with the repository.

Setting	Value
Objects	8
Train sequences	00, 01, 02
Validation sequences	03
Test sequences	04
Total trials	100
Best trial	85
Best validation loss	0.174072

The best hyperparameters reported there are:

loss_render_rgb = 0.9299371264458938
loss_render_depth = 0.10124096582954464
loss_tactile = 0.5466793627189213
loss_surface_constraints = 0.5333992007167064
learning_rate = 0.015630480494192155
optimizer = adamw
weight_decay = 1.646271820594222e-05
lr_decay = 0.9907079600438413

Appendix C

Manipulation Benchmark Summary Placeholder

This appendix section is reserved for the final manipulation-prior benchmark summary after the thesis experiment sweep is complete.

Table C.1: Reserved appendix table for the manipulation-prior benchmark summary.

Model	Runtime (s)	Accuracy summary
H3D	TBD	TBD
TripoSR	TBD	TBD
InstantMesh	TBD	TBD
Geco	TBD	TBD
RGB2Point	TBD	TBD
Fast-SAM3D	TBD	TBD

Appendix D

Selected Implementation Excerpts

The main chapters describe algorithms and design choices at the method level. This appendix records a few short implementation excerpts that correspond to core ideas in the deployed system. The goal is not to reproduce the codebase in the thesis, but to show where several method claims become concrete in the implementation.

D.1 Tactile-Aware Frame-0 Translation Seed

The first excerpt shows the implemented translation seed used when the tracker initializes without ground-truth pose:

Listing D.1: Tracker excerpt for tactile-aware frame-0 translation seeding.

```
if tactile_world is not None and tactile_world.numel() > 0:
    tactile_centroid = tactile_world.mean(0)
    depth_centroid = p_world.mean(0)
    centroid = 0.7 * tactile_centroid + 0.3 * depth_centroid
else:
    centroid = p_world.mean(0)
```

This is the implementation counterpart of Equation (4.2). The code shows that tactile contacts are preferred when available, but the depth cloud still contributes so the initialization does not collapse onto a very small contact patch.

D.2 Occlusion-Aware Modality Reweighting

The next excerpt shows the loss reweighting logic applied when finger occlusion is detected:

Listing D.2: Loss-orchestrator excerpt for occlusion-aware modality weighting.

```
visual_scale = 1.0 - occlusion_ratio * (1.0 - self.min_visual_weight)

adjusted_rgb = base_rgb_weight * visual_scale
adjusted_depth = base_depth_weight * visual_scale
adjusted_tactile = (
    base_tactile_weight
    * (1.0 + occlusion_ratio * self.tactile_boost_factor)
)
```

This snippet is the direct implementation of Equation (4.17). It shows that the system does not switch modalities discretely; instead, it reduces visual supervision continuously while boosting tactile supervision as occlusion grows.

D.3 Frame-0 Hunyuan Variant Search and Alignment

The final excerpt captures the manipulation-side wrapper around Hunyuan3D-2-mini:

Listing D.3: Manipulation-side excerpt for staged Hunyuan variant search.

```
run_variant_batch(base_variants, "cardinal_base")
current_best = max(candidates, key=_candidate_sort_key)

if self.include_flip_branches and not _candidate_is_decisive(current_best):
    run_variant_batch(flip_variants, "cardinal_flip")
    current_best = max(candidates, key=_candidate_sort_key)

if not _candidate_is_decisive(current_best):
```

```
if _candidate_needs_full_diagonal_search(current_best):
    batch_variants = make_h3d_input_variants(...)
else:
    batch_variants = _select_focused_diagonal_variants(...)
run_variant_batch(batch_variants, "diagonal")

best = max(candidates, key=_candidate_sort_key)
best["transform"] = polish_h3d_object_transform(...)
```

This excerpt illustrates the main integration contribution of Chapter 5. The pretrained image-to-3D model is not queried on a fixed exhaustive bank. Instead, the implementation performs a staged search: cardinal rotations first, mirrored branches only if needed, and diagonal variants only when the earlier stages are still ambiguous. The winning branch is then polished before it becomes the manipulation prior.

Appendix E

Policy Output Contract

The manipulation bridge writes the following production artifacts per exported frame:

Table E.1: Files written by the manipulation export bridge.

Filename pattern	Contents
<code>frame_{N:04d}</code> <code>_cloud.npz</code>	points, provenance labels, real/generated subsets, T_{WO}
<code>frame_{N:04d}</code> <code>_pose.npy</code>	object-to-world transform T_{WO}
<code>frame_{N:04d}.ply</code>	optional debug visualization
<code>frame_{N:04d}</code> <code>_timing.json</code>	optional timing breakdown

The bridge-point cloud used by the internal gap-filling stage is not part of this production contract. It is kept inside the manipulation runtime as a temporary stabilization aid and is only visualized through optional debug artifacts.

This appendix summarizes the artifact and configuration structure used by the implemented GaussianFeels pipeline.